

Synthèse sur la complexité O

* Instructions basiques $\leadsto O(1)$

$\leadsto$ Tranche de taille $k \leadsto O(k)$

! Si il faut parcourir une liste $\leadsto O(m)$

Efficacité d'Algo. itératif

$\Rightarrow$ Boucle for :

Création de la séquence + nbr d'itération $\times$ corps de la boucle

$\Rightarrow$ Boucle while :

nbr itérations $\times$ (condit° + corps) + sortie $\approx$ condit°

! Plusieurs boucles imbriquées

INTERNE $\longrightarrow$ EXTERNE

Exemples concrets :

$\rightarrow$ Recherche linéaire : On parcourt la liste m fois

séquence déjà exist° $\leadsto O(1)$ et corps en $O(1)$

$$\rightarrow O(1) + O(m) \times O(1) = O(m)$$

$\rightarrow$ Tri par insertion :

• Boucle interne (while) $\leadsto$ pire des cas : i fois $\Rightarrow O(i)$

• Boucle externe (for) $\leadsto$ exécute $m-1$ fois $\Rightarrow O(m)$

$\Rightarrow$ formule d'Euler

$$O(m) + \sum_{i=0}^{m-1} O(i) = O\left(\sum_{i=1}^m i\right) = O\left(\frac{m^2+m}{2}\right) \Rightarrow O(m^2)$$

$\rightarrow$ Tri par sélection :

• Boucle interne (for) $\leadsto O(m-i)$ parcourue $m-i-1$ fois

• Boucle externe (for) $\leadsto O(m-1)$

$$\begin{aligned} \rightarrow O(m-1) + \sum_{i=0}^{m-2} O(m-i) &= O(m-1) + O(m + (m-1) + \dots + 2) \\ &= O(m-1) + O\left(\frac{m^2+m}{2}\right) = O(m^2) \end{aligned}$$

→ Tri à bulles:

- Boucle interne: $O(m-1) = O(m)$
- Boucle externe: parcourue m fois.
 $\leadsto O(m) \times (O(1) + O(m)) + O(1) = O(m^2)$

Efficacité d'Algo Récursifs

① Trouver une formule inductive pour $T(m)$

- cas de base: $\rightarrow$ on remplace par une constante
- cas général: $\iff$ On trouve la formule non-inductive

Exemple: Somme récursive

Cas de base: $m=1 \leadsto T(1) = O(1)$

Cas général: $T(m) = T(m-1) + O(1) \quad \forall m > 1$

$$\rightarrow T(1) = a \quad T(m) = T(m-1) + b \quad \forall m > 1$$

$$T(1) = a$$

$$T(2) = T(2-1) + b = T(1) + b = a + b$$

$$T(3) = T(2) + b = a + b + b = a + 2b$$

$$T(4) = T(3) + b = a + 2b + b = a + 3b$$

$$T(m) = a + (m-1)b \quad \forall m \geq 1 \rightarrow \text{forme non-inductive}$$

$$\begin{matrix} O(1) & & O(m-1) = O(m) \end{matrix}$$

$$\rightarrow T(m) \in O(1) + O(m) = O(m)$$

→ Tri par fusion:

$$\Rightarrow \text{Split: } O(m_1) + O(m_2) = O(m_1 + m_2) = O(m)$$

$$\Rightarrow \text{Merge: Base: } T(1) = O(1) \quad \text{Gén: } T(m) = T(m-1) + O(1)$$

$$\rightarrow T(1) = a \quad T(m) = T(m-1) + b$$

$$\hookrightarrow T(1) = a \quad T(2) = a + b \quad T(3) = a + 2b$$

$$\Rightarrow T(m) = a + (m-1)b$$

$$\hookrightarrow T(m) \in O(m)$$

⇒ merge-sort : Base : $O(1) = T(1)$

Gen : $T(m) = O(m) + 2T(m/2)$

⇒ on considère m est une puissance de 2 (>1)

$$T(1) = a \quad T(m) = bm + 2T(m/2)$$

$$T(1) = a$$

$$T(2) = 2b + 2T(1) = 2a + 2b$$

$$T(4) = 4b + 2T(2) = 4b + 4a + 4b = 4a + 8b$$

$$T(8) = 8b + 2T(4) = 8a + 24b$$

↳ $T(m) = ma + bm \log_2(m)$

$O(m)$

$O(m \log(m))$

↳ $T(m) \in O(m \log(m))$

* Règles Théoriques :

- On néglige les facteurs constants

$$O(2m) = O(m) \quad \text{ou} \quad O(3) = O(1)$$

- On néglige les termes d'ordre inférieur

$$O(m^2 + m) = O(m^2)$$

- Somme → on prend le maximum $O(m) + O(1) = O(m)$

- Multipliatⁿ → $O(f(m)) \cdot O(g(m)) = O(f(m) \cdot g(m))$

$$O(m) \cdot O(m^2) = O(mm^2) = O(m^3)$$

- Transitivity : $f \in O(g(m)) \wedge g \in O(h(m))$

$$\Rightarrow f \in O(h(m))$$

En bref :

Simplification

Proximité



$$T(m) = 2m^2 + 3 \in O(m^3)$$

ou encore $O(2^m)$ MAIS

le + PROCHE est $O(m^2)$

MÉMO : PROGRA OBJET

→ Ejs définir `--init--` et `--str--`

* `def --init--(self, ~, ~, ...)`

`self.mome = ~`
est l'instance

* pour `--str--(self)` : on met ce qu'on veut voir

`' ~ ' % ( ~ )`
↙ on forme le str les variables utilisées pour former le str

Exemple : `'%d/%d' % (self.x, self.y)`

entier avec autant de digits qu'on veut

↳ `%02d` ⇒ max 2 digits etc etc

⊕ `%02f` ⇒ float et 2 mb après la virgule max

* `--add--` et `--radd--`

→ pour additionner deux obj, un obj et un entier, etc

→ `def --add--(self, other):`

if `isinstance(other, int)`:

`return ~`

→ Permet de savoir si `other` est de type entier (ou autres)

⊕ Si on veut faire `other + self` et pas l'inverse:

`def --radd--(self, other):`

`return self.--add--(self, other)`

↳ on ajoute juste `other` à `self` (donc `self + other`)

car l'addition est commutative

recherche - dichotomique

⇒ Avant la moitié on retourne avec `[:mid]`

↳ cas de base

`len == 1`

`< : 0`

`> : 1`

Après la moitié on retourne `mid + [mid:]`